

BUILD-SUPABASE.md

Partie C.13 — Sécurité PostgreSQL

Version : V1

Statut : Référence officielle de la sécurité PostgreSQL

1. Objectif

Cette section définit les règles officielles de sécurité PostgreSQL pour Vitala.

Les objectifs sont :

- protéger les données ;
- garantir la confidentialité ;
- assurer l'intégrité ;
- limiter les accès non autorisés ;
- appliquer le principe du moindre privilège.

La sécurité est intégrée dès la conception de la base de données.

2. Principes directeurs

Toute la sécurité PostgreSQL repose sur les principes suivants :

- Security by Design
- Zero Trust
- Least Privilege
- Defense in Depth
- Auditabilité
- Documentation obligatoire

Aucun accès ne doit être considéré comme implicitement autorisé.

3. Authentification

L'authentification des utilisateurs est assurée par Supabase Auth.

PostgreSQL ne gère pas directement les comptes utilisateurs de l'application.

Les rôles techniques sont réservés à l'infrastructure.

4. Autorisation

Les permissions doivent être accordées uniquement lorsque nécessaires.

Chaque rôle dispose uniquement des privilèges indispensables à sa mission.

Les privilèges inutilisés doivent être supprimés.

5. Row Level Security (RLS)

La **Row Level Security (RLS)** est obligatoire sur toutes les tables contenant des données métier.

Chaque table doit définir :

- les politiques de lecture ;
- les politiques d'insertion ;
- les politiques de modification ;
- les politiques de suppression.

Aucune table métier ne peut être exposée sans politique RLS.

6. Politiques RLS

Les politiques doivent être :

- simples ;
- explicites ;
- documentées ;
- testées.

Les politiques complexes doivent être décomposées afin de rester compréhensibles.

7. Colonnes sensibles

Les informations sensibles doivent bénéficier d'une protection renforcée.

Exemples :

- données personnelles ;
- coordonnées privées ;
- informations de confiance ;
- éléments de sécurité.

Lorsque nécessaire, ces informations doivent être chiffrées ou masquées.

8. Fonctions sécurisées

Les fonctions SQL doivent utiliser :

- `SECURITY INVOKER` par défaut.

`SECURITY DEFINER` n'est autorisé que lorsque :

- le besoin est démontré ;
 - le risque est évalué ;
 - la documentation est complète.
-

9. Vues sécurisées

Les vues destinées à exposer des données doivent :

- masquer les colonnes sensibles ;
- respecter les politiques RLS ;
- limiter les informations retournées.

Les vues ne doivent jamais contourner les règles de sécurité.

10. Journalisation

Les opérations sensibles doivent être enregistrées.

L'audit doit permettre de tracer :

- les accès critiques ;
- les modifications importantes ;
- les suppressions ;
- les opérations administratives.

Les journaux doivent respecter les politiques de conservation définies par Vitala.

11. Chiffrement

Les données sensibles doivent être protégées :

- en transit (TLS) ;
- au repos (mécanismes fournis par Supabase) ;

- au niveau applicatif lorsque le contexte l'exige.

Les secrets ne doivent jamais être stockés en clair dans la base.

12. Secrets

Les clés API, jetons d'accès et secrets applicatifs doivent être stockés dans des mécanismes dédiés.

Ils ne doivent jamais être enregistrés dans les tables métier.

13. Surveillance

La sécurité PostgreSQL doit être surveillée afin de détecter :

- les tentatives d'accès non autorisées ;
- les erreurs répétées ;
- les comportements anormaux ;
- les opérations administratives inhabituelles.

Des alertes doivent être mises en place pour les événements critiques.

14. Tests de sécurité

Les politiques de sécurité doivent être testées régulièrement.

Les tests doivent vérifier :

- l'application correcte des politiques RLS ;
 - les permissions des rôles ;
 - l'absence d'accès non autorisés ;
 - la résistance aux erreurs de configuration.
-

15. Documentation

Toute règle de sécurité doit être documentée avec :

- son objectif ;
 - les tables concernées ;
 - les rôles concernés ;
 - les politiques appliquées ;
 - les impacts éventuels.
-

16. Évolution

Toute modification des règles de sécurité doit :

- être réalisée via une migration lorsque nécessaire ;
 - être testée ;
 - être validée avant la mise en production ;
 - mettre à jour la documentation.
-

17. Critères de conformité

La sécurité PostgreSQL est conforme lorsque :

- toutes les tables métier utilisent RLS ;
 - les permissions respectent le principe du moindre privilège ;
 - les fonctions sont sécurisées ;
 - les données sensibles sont protégées ;
 - les règles sont documentées ;
 - les tests de sécurité sont validés.
-

18. Conclusion

La sécurité PostgreSQL constitue le dernier rempart protégeant les données de Vitala.

Une application rigoureuse des politiques d'accès, de la Row Level Security, de l'audit et du principe du moindre privilège garantit une plateforme robuste, fiable et conforme aux exigences du Build Blueprint.